

# Assignment 2: PulseFit — Fitness & Workout Tracker

Pending



## Objective

Build a **FastAPI-based Fitness & Workout Tracker API** where users can log workouts, track their fitness activity, and view only their own workout data.

The assignment should demonstrate:

- JWT Authentication
- Dependency Injection
- Current User handling
- Enum-based validation
- Date-time handling
- Optional fields
- Pagination
- User-specific data access

---

## Technical Requirements

- **Framework:** FastAPI
- **Authentication:** JWT-based login
- Use `Depends(get_current_user)` for protected routes
- Use `datetime.utcnow()` for workout logs
- Pagination defaults:
  - `limit = 10`
  - `offset = 0`

---

## Workout Types

## workout types

Use an Enum with the following values:

- cardio
- strength
- yoga
- hiit
- sports

The user's **fitness level** should also be implemented using an appropriate Enum.

---

## Database Models

### User

- id
- email
- password
- fitness\_level

### Workout

- id
  - user\_id
  - type
  - duration
  - calories\_burned
  - created\_at
- 

## API Routes

POST /users/signup

Register a new user with:

- Email
- Password
- Fitness level

Fitness level must be validated using an Enum.

---

### **POST /workouts**

Log a new workout for the authenticated user.

Required fields:

- Workout type
- Duration
- Calories burned

Must use:

Depends(`get_current_user`)

The logged workout must automatically belong to the authenticated user.

---

### **GET /workouts**

Fetch the authenticated user's workout history.

Support pagination using:

`limit = 10`

`offset = 0`

Users must only be able to view **their own workout records**.

---

### **GET /workouts/stats**

Return workout statistics for the authenticated user.

Calculate:

- **Total calories burned**

The calculation logic should be implemented using a **FastAPI Dependency**.

---

## **DELETE /workouts/{id}**

Delete a workout record.

Requirements:

- Users can delete only their own workouts.
  - Return 404 Not Found if the workout ID does not exist.
  - Users must not be allowed to delete another user's workout.
- 

## **Validation Rules**

- `duration > 0`
- `calories_burned >= 0`
- Workout type must be a valid Enum
- Fitness level must be a valid Enum
- Workout `created_at` should use `datetime.utcnow()`
- Users can access only their own workout data

Use Pydantic validation wherever applicable.

Example:

```
duration: int = Field(gt=0)
calories_burned: float = Field(ge=0)
```

---

## **Authentication & Dependency Injection**

Protected workout routes must identify the logged-in user through:

`Depends(get_current_user)`

The authenticated user's ID should be used when:

- Creating workouts
- Viewing workout history

- viewing workout history
  - Calculating statistics
  - Deleting workouts
- 

## Pagination

GET /workouts must support:

limit  
offset

Default values:

limit = 10  
offset = 0

---

## Error Handling

Handle appropriate errors for:

- Invalid login
- Invalid or expired JWT
- Unauthorized access
- Workout not found
- Invalid workout type
- Invalid fitness level
- Invalid duration
- Negative calories
- Attempt to access another user's workout

Use appropriate HTTP status codes such as 400, 401, 403, 404, and 422.

---

## API Testing

All APIs should be testable through FastAPI Swagger:

/docs

Use the **Authorize** option to test JWT-protected APIs.

---

# Submission Instructions

Submit the **complete project as ONE ZIP file through the LMS.**

The ZIP file should include:

- Complete source code
- requirements.txt
- README.md
- All files required to run the project

## ZIP File Name

StudentName\_PulseFit.zip

Example:

RahulKumar\_PulseFit.zip

## Important

- Submit only **one ZIP file**.
  - Do not upload individual files separately.
  - Do not include `venv`, `.venv`, `__pycache__`, or other unnecessary folders.
  - Test the application before submission.
  - Ensure the correct ZIP file is uploaded successfully to the LMS.
  - Submit before the deadline mentioned in the LMS.
- 

## Evaluation Criteria

The assignment will be evaluated based on:

- Correct API functionality

- JWT Authentication
- Dependency Injection
- Current User implementation
- User-specific data access
- Enum validation
- Date-time handling
- Pagination
- Error handling
- Code quality and organization
- Successful project execution
- Proper ZIP submission

## Expected Outcome

By completing this assignment, you should be able to build a secure **FastAPI application with JWT authentication, dependency injection, user-specific data access, validation, pagination, and workout tracking functionality.**

Upload Assignment